

数论算法

快速幂

```
int qmi(int m, int k, int p)
{
    int res = 1 % p, t = m;
    while (k)
    {
        if (k&1) res = res * t % p;
        t = t * t % p;
        k >>= 1;
    }
    return res;
}
```

乘法逆元

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;

ll fpm(ll x, ll power, ll mod) {
    x %= mod;
    ll ans = 1;
    for (; power; power >>= 1, (x *= x) %= mod)
        if(power & 1) (ans *= x) %= mod;
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,p;
    cin>>n>>p;
    for(int a=1;a<=n;a++){
        cout<< fpm(a, p - 2, p)<<endl;
    }//x为a在mod p意义下的逆元
}
```

组合数求法

```
#include <iostream>

using namespace std;

typedef long long LL;

LL qmi(LL a,int k, int p)
{
    LL res = 1;
    while(k)
    {
        if (k & 1) res = res * a % p;
        k >>= 1;
        a = a * a % p;
    }
    return res;
}

int C(int a,int b,int p)
{
    if(a < b) return 0;

    LL res = 1;
    for(int i = 1, j = a; i <= b; i ++ , j -- )
    {
        res = res * j % p;
        res = res * qmi(i, p - 2, p) % p;
    }
    return res;
}

LL lucas(LL a, LL b, int p)
{
    if(a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}

int main()
{
    int n;
    cin >> n;
```

```
while(n --)
{
    LL a,b;
    int p;
    cin >> a >> b >> p;
    cout << lucas(a, b, p) << endl;
}
return 0;
}
```

线性基

//普通消元法

```
void insert(long long x)
{
    for(int i=63;~i;i--) if((x>>i)&1)
    {
        if(!place[i]) {place[i]=x;return;}
        else x^=place[i];
    }
}

void insert(int *num)
{
    for(int j=63;~j;j--)
    {
        int p=0;
        for(int i=1;(!p)&&i<=n;i++)
            if((!v[i])&&(num[i]>>j)&1) p=i;
        if(!p) {place[j]=0;continue;}
        v[p]=true,place[j]=num[p];
        for(int i=1;i<=n;i++) if(i!=p&&(num[i]>>j)&1)
            num[i]^=num[p];
    }
}

bool check(long long x)
{
    for(int i=63;~i;i--)
        if((x>>i)&1) x^=place[i];
    return x==0;
}

struct Base
{
    long long place[N];
    void insert(long long x)
    {
        for(int i=63;~i;i--) if((x>>i)&1)
        {
            if(!place[i]) {place[i]=x;return;}
            else x^=place[i];
        }
    }
};

void merge(Base A,Base &B)
```

```
{
    for(int i=63;~i;i--) if(A.place[i])
        B.insert(A.place[i]);
}
long long mx()
{
    long long ans=0;
    for(int i=63;~i;i--)
        if((ans^place[i])>ans)
            ans^=place[i];
    return ans;
}
long long mn()
{
    for(int i=0;i<=63;i++)
        if(place[i]) return place[i];
    return -1;
}
```

```

//K大异或和
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
typedef long long ll;
ll b[55];
int flg;
void insert(ll x) {
    int i;
    for(i=50;i>=0;i--) {
        if(x&(1ll<<i)) {
            if(b[i]) x^=b[i];
            else {b[i]=x; return ;}
        }
    }
    if(!x) flg=1;
}
void Guass() {
    int i,j;
    for(i=50;i>=0;i--) {
        if(b[i]) {
            for(j=50;j>=0;j--) {
                if(i!=j&&(b[j]&(1ll<<i))) {
                    b[j]^=b[i];
                }
            }
        }
    }
}
int main() {
    int n,m;
    scanf("%d",&n);
    int i;
    ll x;
    for(i=1;i<=n;i++) scanf("%lld",&x),insert(x);
    Guass();
    int cnt=0;
    for(i=0;i<=50;i++) {
        if(b[i]) b[cnt++]=b[i];
    }
    scanf("%d",&m);
    while(m--) {

```

```
scanf("%lld",&x);
if(x>(111<<cnt)) {
    puts("-1"); continue;
}
x-=flg;
ll ans=0;
for(i=cnt;i>=0;i--) {
    if(x&(111<<i)) ans^=b[i];
}
printf("%lld\n",ans);
}
}
```

矩阵快速幂及高斯消元

```
// 1.矩阵快速幂
#include <bits/stdc++.h>
using namespace std;
#define int long long
using ll = long long;
const int MOD = 1e9 + 7;
// 矩阵结构体
struct Matrix {
    int n, m; // n: 行数, m: 列数
    vector<vector<ll>> a;
    Matrix(int _n, int _m) : n(_n), m(_m), a(_n + 1, vector<ll>(_m + 1, 0)) {}

    static Matrix identity(int n) {
        Matrix I(n, n);
        for (int i = 1; i <= n; ++i) {
            I.a[i][i] = 1;
        }
        return I;
    }
};

// 矩阵乘法: C = A * B
// 时间复杂度: O(n^3) 或 O(A.n * A.m * B.m)
Matrix operator*(const Matrix& A, const Matrix& B) {
    Matrix C(A.n, B.m);
    for (int i = 1; i <= A.n; ++i) {
        for (int j = 1; j <= B.m; ++j) {
            for (int k = 1; k <= A.m; ++k) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

// 矩阵快速幂: A^p, 时间复杂度: O(n^3 * log p), A必须是方阵
Matrix power(Matrix A, ll p) {
    Matrix res = Matrix::identity(A.n);
    while (p) {
        if (p & 1) res = res * A;
        A = A * A;
        p >>= 1;
    }
}
```

```

    }
    return res;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int n, k; cin >> n >> k;
    Matrix A(n, n);
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cin >> A.a[i][j];
        }
    }
    Matrix res = power(A, k);
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cout << res.a[i][j] << " ";
        }
        cout << endl;
    }
}

```